

XIAMEN UNIVERSITY

COM102 – Programming Language C

Chapter 5 – **Pointers and Arrays** (i) – Lab

Dr. Najla'a

Task 1

- **Input and Print a One-Dimensional Array**
- **Objective:**
- Write a program that defines a one-dimensional array of integers, takes input from the user to populate the array, and then prints the elements of the array.
- **Instructions:**
  1. **Declare a one-dimensional array of integers** with a size specified by the user.
  2. **Prompt the user to input values** for each element of the array.
  3. **Print the elements of the array** along with their indices.

Task 1 - Solution

```c
#include <stdio.h>

int main() {
    int size;

    // Prompt user to enter the size of the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &size);

    int arr[size]; // Declare the array with the specified size

    // Input elements into the array
    printf("Enter %d integers:\n", size);
    for (int i = 0; i < size; i++) {
        printf("Element [%d]: ", i);
        scanf("%d", &arr[i]);
    }

    // Print the elements of the array
    printf("The elements of the array are:\n");
    for (int i = 0; i < size; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    return 0;
}
```

Task 2

Search for a Specific Value in the Array

Objective:

Implement a function that searches for a specific value entered by the user in the one-dimensional array defined in Task 1. Return and print the index of the value if found; otherwise, print a message indicating that the value is not in the array.

Instructions:

1. **Define a function** called *searchInArray* that takes three parameters:
   - The integer array to search in.
   - The size of the array.
   - The target value to search for.
2. In the function, **iterate through the array** to check if the target value is present.
3. If the value is found, **return the index**; otherwise, return -1.
4. In the main function, **call the search function** and print the result based on its return value.

Task 2 - Solution

```c
#include <stdio.h>

// Function to search for a specific value in a 1D array
int searchInArray(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Return the index if found
        }    }
    return -1; // Return -1 if not found
}
int main() {
    int size;
    // Prompt user to enter the size of the array
    printf("Enter the number of elements in the array: ");
    scanf("%d", &size);
    int arr[size]; // Declare the array with the specified size
    // Input elements into the array
    printf("Enter %d integers:\n", size);
    for (int i = 0; i < size; i++) {
        printf("Element [%d]: ", i);
        scanf("%d", &arr[i]);}
    // Input the target value to search for
    int target;
    printf("Enter the value to search for: ");
    scanf("%d", &target);

    // Call the search function and store the result
    int index = searchInArray(arr, size, target);

    // Print the result
    if (index != -1) {printf("Element %d found at index %d\n", target, index);}
    else {        printf("Element %d not found in the array\n", target);    }

    return 0;
}
```

Task 3

Objective: Enhance your program by implementing additional functions to improve its functionality.

Instructions:

1. **Max Function**:
   - Develop a function named findMax that takes an integer array and its size as parameters.
   - The function should iterate through the array to find and return the maximum value present in it.
2. **Min Function**:
   - Implement a function named findMin that also takes an integer array and its size as parameters.
   - This function should traverse the array to identify and return the minimum value found within.

Task 3 - Solution

```
}
int findMax(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

int findMin(int arr[], int size) {
    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}
```

Task 4

Objective:

Read n characters into an array and check whether any character is a space ' '.
If yes, print the index (or indices) where spaces appear; otherwise print "No space found."

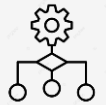
Constraints:

- $1 \leq n \leq 100$
- Input characters **exactly** (including spaces).
- Use only main, loops, and basic variables.

Task 4 - Solution

```c
#include <stdio.h>
int main(void) {
    int n, i;
    printf("Enter number of characters (1..100): ");
    scanf("%d", &n);getchar();
    char a[n];
   // Input exactly n characters (including spaces/newlines if typed)
    printf("Enter %d characters (spaces are allowed):\n", n);
    for (i = 0; i < n; i++) {
        scanf("%c", &a[i]);
        getchar();
          }
    // Detect spaces and report indices
    int found = 0;
    printf("Space found at indices: ");
    for (i = 0; i < n; i++) {
        if (a[i] == ' ') {
            printf("%d ", i);
            found = 1;
        }
    }
    if (!found) {printf("None (No space found)");}
    printf("\n");
    return 0;
}
```

Task 5: Reverse the Array (In-Place)



Objective:
Read n integers into an array, **reverse** the array *in place* (by **swapping**), then **print** the reversed array with indices.

Constraints:

- 1 ≤ n ≤ 100
- Use only main, loops, and basic variables (no functions, no pointers).

Example I/O
Input: n=5, array: 10 20 30 40 50
Output indices/values after reverse:
0: 50
1: 40
2: 30
3: 20
4: 10

```c
#include <stdio.h>
int main(void) {
    int n, i;
    printf("Enter array size (1..100): ");
    scanf("%d", &n);
    int a[n];
    // Input
    for (i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }

    // In-place reverse
    for (i = 0; i < n / 2; i++) {
        int tmp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = tmp;
    }
    // Output
    printf("\nReversed array:\n");
    for (i = 0; i < n; i++) {
        printf("%d: %d\n", i, a[i]);
    }
    return 0;
}
```

Array Programming Exercises

Exercise 1

- **Max and Min in 2D Array**
  - **Task**: Create a program that initializes a two-dimensional array with integer values. Implement a function to find and return the maximum and minimum values in the array. Print the results in the main function.

Exercise 2

Array Reversal

- **Task**: Implement a program that reverses a one-dimensional array. Write a function that takes an array and its size as parameters and returns the reversed array. Print the original and reversed arrays in the main function.

Exercise 3

- **Count Even and Odd in 1D Array**
  - **Task**: Write a program that initializes an array of integers with at least 10 elements. Implement a function to count and return the number of even and odd numbers in the array. Print the counts in the main function.

Exercise 4

- **Transpose of a 2D Array**
  - **Task**: Create a program that initializes a 2D array (e.g., 3x3) with integer values. Implement a function to calculate and print the transpose of the array.

Exercise 5

- **Finding Duplicate Values in 1D Array**
  - **Task**: Write a program that allows the user to input 10 integers into a one-dimensional array. Implement a function to identify and print any duplicate values in the array.

Exercise 6

- **Sorting a 1D Array**
  - **Task**: Create a program that initializes a one-dimensional array of integers. Implement the bubble sort algorithm in a function to sort the array in ascending order. Print the sorted array in the main function.

Exercise 7

- **Average of 2D Array Elements**
  - **Task**: Write a program that initializes a two-dimensional array with integer values (e.g., 4x3). Implement a function to calculate and return the average of all the elements in the array. Print the average in the main function.

Exercise 8

- **Array Rotation**
  - **Task**: Implement a program that rotates a one-dimensional array to the right by a given number of positions. Write a function that takes the array, its size, and the number of positions to rotate. Print the rotated array in the main function.

Exercise 9

Diagonal Sum of a Two-Dimensional Array

Objective:

Create a program that calculates the sum of the diagonal elements of a square two-dimensional array (matrix). This exercise allows you to work with square matrices and focus on specific elements.

Instructions:

1. **Declare a square two-dimensional array** of size n x n, where n is specified by the user.
2. **Prompt the user** to enter the elements of the array.
3. **Implement a function** that calculates and returns the sum of the diagonal elements.
4. **Print the sum of the diagonal elements**

Exercise 10

Unique Elements in a One-Dimensional Array

Objective:

Write a program that identifies and prints all unique elements in a one-dimensional array. This exercise will help you understand how to process arrays and utilize nested loops for comparison.

Instructions:

1. **Declare a one-dimensional array** of integers with a size specified by the user.
2. **Prompt the user** to enter the elements of the array.
3. Implement a function that takes the array and its size as parameters and identifies unique elements (those that appear exactly once in the array).
4. **Print the unique elements** in the main function.

Exercise 11

Matrix Addition

Objective:

Write a program that performs the addition of two square two-dimensional arrays (matrices) and outputs the resulting matrix. This exercise will help you understand how to work with 2D arrays and implement basic matrix operations.

Instructions:

1. **Declare two square two-dimensional arrays** of size n x n, where n is specified by the user.
2. **Prompt the user** to enter the elements of both matrices.
3. Implement a function that takes the two matrices and their size as parameters, performs the addition, and stores the result in a third matrix.
4. **Print the resulting matrix** in the main function